CS771 : Introduction to Machine Learning

Minor Assignment-2

Aarsh Jain
230015
aarshjain23@iitk.ac.in

Aayush Kumar
230027
aayushk23@iitk.ac.in

Bikramjeet Singh
230298
bsingh23@iitk.ac.in

Hitarth Makawana
230479
hitarthkm23@iitk.ac.in

Kulshreshth Chikara
230586
kulshresh23@iitk.ac.in

Ronav Puri
230815
ronavgp23@iitk.ac.in

This report contains the solutions to Minor Assignment-2 of the course CS771 2025-26 I. It presents the extension of a Gaussian generative classifier (based on a single Gaussian per class) to a Gaussian Mixture Model (GMM)-based classifier for the MNIST dataset. Each digit class is modeled as a mixture of K Gaussians (parameterized), and the experiments analyze the effects of varying K on classification accuracy and reconstruction capability in the presence of missing pixels.

For the various derivations related to EM algorithm especially the reconstruction part and manipulations to the code we referred to some online resources on the same links to whom have been attached at the end of the document.

1 Introduction

Generative models aim to model the joint distribution $P(x, y)$ and use Bayes' rule to predict class labels for unseen data.The lecture code provided models for each class using a single Gaussian distribution. To capture variations in each class (e.g., different handwriting styles of the same digit), we generalize the model using Gaussian Mixture Models (GMMs). This is similar in theory to having multiple centroids/representatives for each class in K-Means.

The report has been structured as follows :

- Section 2 outlines detailed calculations for training a GMM-based generative multi-classification model.
- Section 3 outlines the inference (testing) procedure for classification of a new point.
- Section 4 outlines the code modifications and results for performing generative classification on uncensored test images from MNIST dataset.
- Section 5 outlines the code modifications and results for performing generative classification and reconstruction in the presence of missing pixels.
- Section 6 outlines the solutions to the bonus questions involving varying K for different classes.

2 Training a GMM-based Generative Multi-Classification Model

We model each class-conditional distribution $P(\mathbf{x} \mid y = c)$ as a mixture of K Gaussian components. Formally, for every class $c \in \{1, 2, \ldots, C\}$,

$$P(\mathbf{x} \mid y = c) = \sum_{k=1}^{K} \pi_k^c \, \mathcal{N}(\mathbf{x};\, \boldsymbol{\mu}_k^c,\, \boldsymbol{\Sigma}_k^c)\,, \tag{1}$$

where

- $\pi^c = [\pi_1^c, \pi_2^c, \ldots, \pi_K^c]$ are the selection probabilities of each of the K components satisfying $\pi_k^c > 0, \sum_k \pi_k^c = 1$ for the cth class;
- $\boldsymbol{\mu}_k^c \in \mathbb{R}^d$ and $\boldsymbol{\Sigma}_k^c \in \mathbb{R}^{d\times d}$ denote the mean and covariance of the kth Gaussian component.

The Gaussian density is given by

$$\mathcal{N}(\mathbf{x}; \boldsymbol{\mu}, \boldsymbol{\Sigma}) = \frac{1}{(2\pi)^{d/2}|\boldsymbol{\Sigma}|^{1/2}} \exp\left[-\tfrac{1}{2}(\mathbf{x}-\boldsymbol{\mu})^\top \boldsymbol{\Sigma}^{-1}(\mathbf{x}-\boldsymbol{\mu})\right]. \tag{2}$$

The prior of each class is estimated empirically as

$$P(y = c) = \frac{N_c}{N}, \tag{3}$$

where N_c is the number of samples in class c and N is the total number of training examples.

2.1 Introduction of Latent Variables

As mentioned in class notes, for GMM formulation, we introduce latent variables $z_c^i \in \{1, \ldots, K\}$ indicating the mixture component that generated data point $\mathbf{x}^i$ of class c. The joint density for a data point $\mathbf{x}^i$ belonging to the cth class and generated by the kth mixture component is given by

$$P(\mathbf{x}^i, z_c^i = k \mid y = c) = \pi_k^c \, \mathcal{N}\big(\mathbf{x}^i;\, \boldsymbol{\mu}_k^c,\, \boldsymbol{\Sigma}_k^c\big), \tag{4}$$

where π_k^c denotes the selection probability of the kth component within the cth class, satisfying $\sum_{k=1}^K \pi_k^c = 1 \;\&\; \pi_k^c >= 0$.

Note: In the lecture notes and slides, for simplicity, it was often assumed that each mixture component was *equally likely*, i.e.

$$\pi_k^c = \frac{1}{K}, \quad \forall\, k \in \{1, 2, \ldots, K\}. \tag{5}$$

Under this uniform assumption, the selection probabilities π_k^c do not explicitly appear in the intermediate EM derivations. However, in this implementation, we relax that assumption and estimate the π_k^c parameters directly from the data during the M-step (as demanded by the assignment).

The marginal log-likelihood for all points belonging to class c is simply summation over all individual likelihoods of data points belonging to the class c (assuming all data points are generated independently)

$$\mathcal{L}_c(\Theta^c) = \sum_{i:y_i=c} \log\left(\sum_{k=1}^K \pi_k^c \, \mathcal{N}(\mathbf{x}^i; \boldsymbol{\mu}_k^c, \boldsymbol{\Sigma}_k^c)\right), \tag{6}$$

which is maximized using the *Expectation–Maximization (EM)* algorithm. Here Θ^c denotes collectively all the individual Gaussian models.

2.2 Expectation–Maximization Algorithm

For each class c, EM alternates between two steps:

E-step: In our setup, the **E-step** of the EM algorithm is implemented using a simplified approximation. Rather than computing the exact probabilities of how likely are different z_i values for the *i*-th data point,

$$q_k^i = \frac{\pi_k^c \, \mathcal{N}(\mathbf{x}^i; \boldsymbol{\mu}_k^c, \boldsymbol{\Sigma}_k^c)}{\sum_{j=1}^K \pi_j^c \, \mathcal{N}(\mathbf{x}^i; \boldsymbol{\mu}_j^c, \boldsymbol{\Sigma}_j^c)},$$

the provided helper routine (`doEStep`) approximates this expression by using the squared Euclidean distance between the data point and each component mean:

$$q_k^i \propto \exp\big(-\tfrac{1}{2}\|\mathbf{x}^i - \boldsymbol{\mu}_k^c\|^2\big).$$

This simplification effectively assumes $\boldsymbol{\Sigma}_k^c \approx \mathbf{I}_d$, reducing the computational cost by avoiding matrix inversions and determinant computations. This approximation greatly accelerates convergence when working with high-dimensional data such as MNIST dataset (784 dimensions per data point in this case). This approximation was given in the code provided.

M-step: Update selection probabilities, means, and covariances by maximizing the expected log-likelihood:

$$N_k^c = \sum_i q_k^i, \qquad \pi_k^c = \frac{N_k^c}{N_c}, \tag{7}$$

$$\boldsymbol{\mu}_k^c = \frac{1}{N_k^c} \sum_i q_k^i \, \mathbf{x}^i, \tag{8}$$

$$\boldsymbol{\Sigma}_k^c = \frac{1}{N_k^c} \sum_i q_k^i (\mathbf{x}^i - \boldsymbol{\mu}_k^c)(\mathbf{x}^i - \boldsymbol{\mu}_k^c)^\top + \lambda I, \tag{9}$$

where λI is a small diagonal regularizer ensuring positive-definite covariance matrices.

The provided implementation (an approximated one) updates means iteratively and computes covariances once after convergence for efficiency.

3 Inference and Testing

Given a test data point $\mathbf{x}$, the goal of the inference step is to predict the most likely class label $\hat{y}$. According to Bayes' rule, we have the probability of a point belonging to a class c (the posterior),

$$P(y = c \mid \mathbf{x}) = \frac{P(\mathbf{x} \mid y = c) P(y = c)}{P(\mathbf{x})} \propto P(\mathbf{x} \mid y = c) P(y = c),$$

Each class c is modeled by a Gaussian mixture model (GMM) consisting of K components:

$$P(\mathbf{x} \mid y = c) = \sum_{k=1}^{K} \pi_k^c \, \mathcal{N}(\mathbf{x} \mid \boldsymbol{\mu}_k^c, \boldsymbol{\Sigma}_k^c),$$

where π_k^c is the selection probability, and $\mathcal{N}(\mathbf{x} \mid \boldsymbol{\mu}_k^c, \boldsymbol{\Sigma}_k^c)$ denotes the multivariate Gaussian density with mean $\boldsymbol{\mu}_k^c$ and covariance $\boldsymbol{\Sigma}_k^c$ (essentially the model approximated using the EM algorithm).

The prior $P(y = c)$ represents the relative frequency of class c in the training set, typically estimated as

$$P(y = c) = \frac{N_c}{N},$$

where N_c is the number of samples in class c and N is the total number of training samples (also learnt during EM step).

The overall prediction is made by maximizing the log posterior:

$$\hat{y} = \arg\max_c \big[\log P(\mathbf{x} \mid y = c) + \log P(y = c) \big].$$

4 Code Modifications and Implementation Details

The code implementation from the lecture modeled each class $c \in \{0, 1, \ldots, 9\}$ using a *single multivariate Gaussian* with parameters $(\boldsymbol{\mu}^c, \boldsymbol{\Sigma}^c)$. Extending this model to accommodate the variations in the writing styles of a single digit, we replace each single Gaussian with a Gaussian Mixture Model (GMM) comprising K components:

$$P(\mathbf{x} \mid y = c) = \sum_{k=1}^{K} \pi_k^c \, \mathcal{N}(\mathbf{x} \mid \boldsymbol{\mu}_k^c, \boldsymbol{\Sigma}_k^c),$$

where π_k^c are non-negative selection probabilities satisfying $\sum_k \pi_k^c = 1$. The following summarises the **major modifications** made to the original code:

1. **Learning Function Update.** The original `learnClassConditionalDist()` computed one mean and covariance per class using closed-form MLE (which is essentially a deterministic solution, though a simple one in terms of its classification capabilities). We modified it in the following manner to accommodate multiple Gaussians per class:
   - The component means were initialized using the already provided function `doKMPPInitGreedy()` which essentially helps to reach convergence faster (very similar to how means are initialised in K-Means++), and covariances were set to identity matrices using `getSigmaInit()` .
   - The EM algorithm was implemented via the helper function `doEMGMM()` to iteratively update the respective q_k^i values for each k belonging to every data point i (E-step) and subsequently the means and covariances for each Gaussian k for each class c were updated, based on the updated q_k^i values from the E-step (M-step).
   - Compute selection probabilities $\pi_k^c = \frac{N_k^c}{\sum_j N_j^c}$ from soft assignments q_k^i returned by EM.
   - Added a small diagonal regulariser λI to each covariance to ensure numerical stability.

2. **Model Representation.** Instead of storing a single pair $(\boldsymbol{\mu}, \boldsymbol{\Sigma})$ per class, each class now maintains arrays of component parameters per class c :

$$\{\pi_k^c, \boldsymbol{\mu}_k^c, \boldsymbol{\Sigma}_k^c\}_{k=1}^K.$$

   We have to learn the per component selection probabilities π_k^c here since we have not assumed them to be equal contrary to the lecture notes where it was assumed to be equal for every model.

3. **Prediction Function Update.** The original `predictGen()` evaluated only one Gaussian per class. It was modified to compute:

$$\log P(\mathbf{x} \mid y = c) = \log \sum_{k=1}^{K} \pi_k^c \mathcal{N}(\mathbf{x} \mid \boldsymbol{\mu}_k^c, \boldsymbol{\Sigma}_k^c),$$

   The above result is simply a combined result using total probability and Bayes' theorem. The predicted class is then obtained via:

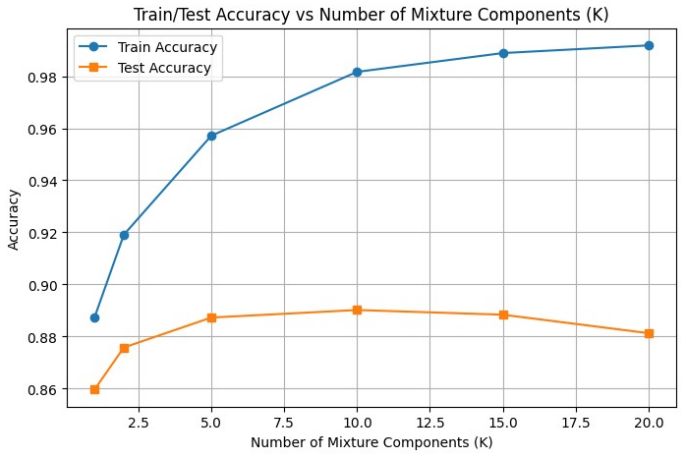
$$\hat{y} = \arg\max_c \big[\log P(\mathbf{x} \mid y = c) + \log P(y = c)\big].$$

4. **Integration of EM Helper Functions.** The EM algorithm was implemented using the provided helper routines:
   - `doKMPPInitGreedy()` for greedily initializing the component means just like K-Means++.
   - `doEStep()` to compute soft responsibilities $q_k^i \propto \exp(-\frac{1}{2}\|\mathbf{x}^i - \boldsymbol{\mu}_k^c\|^2)$, approximating $\boldsymbol{\Sigma}_k^c \approx I$ for avoiding complex computations involving matrix multiplications and inverses (essentially the E-step).
   - `doMStep()` updating of component means essentially the M-step.
   - `doEMGMM()` to iteratively alternate these two steps and perform a final covariance update using weighted MLE (only one updation as mentioned in the provided notebook to accelerate the process).

Overall, these changes extend the original generative classifier from a unimodal Gaussian per class to a multimodal mixture model, enabling better modeling of intra-class variability in the MNIST digits.

4.1 Accuracy Trends with Increasing K

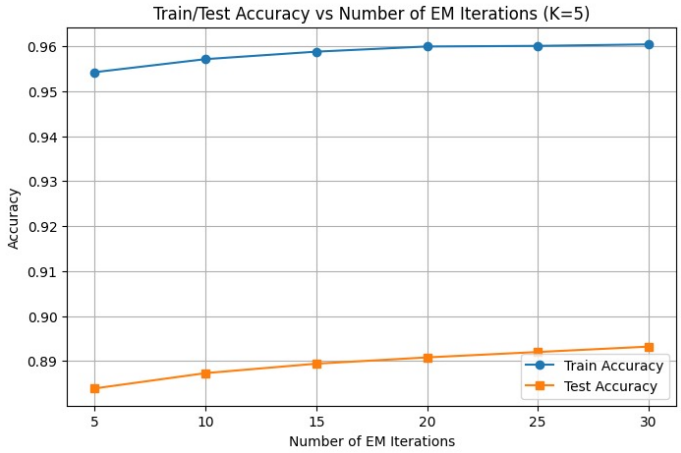
The following plot shows the variation of train and test accuracy as K increases.

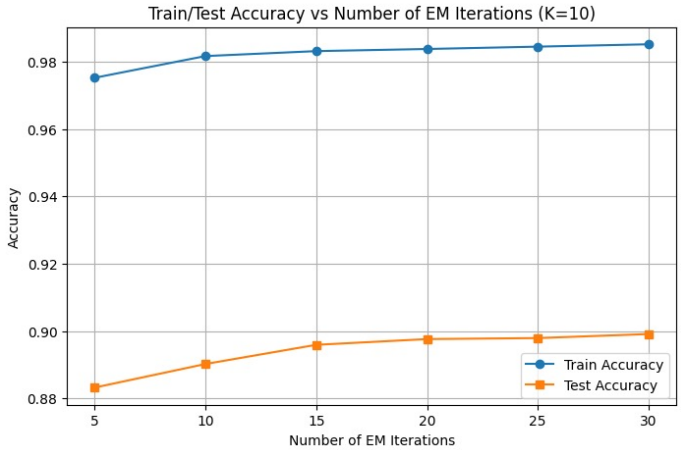


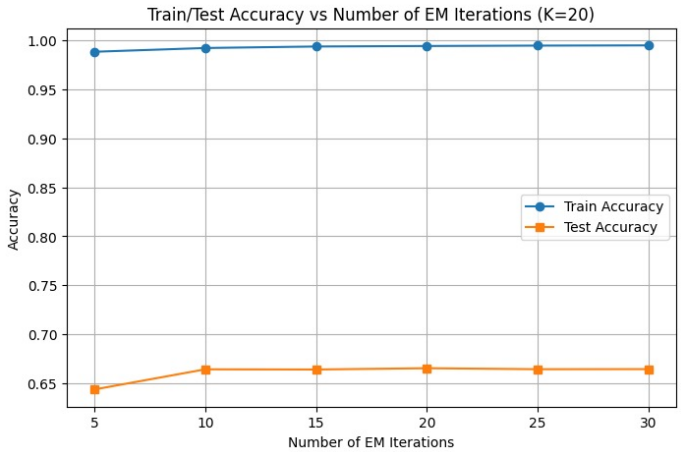
The accuracy improves as K increases, indicating that additional mixture components better capture intra-class variations. After $K = 10$, the accuracy saturates, showing diminishing returns likely due to overfitting on the training data and essentially performing poor on the test data.

4.2 Accuracy Trends with varying number of iterations

The following plot shows the variation of train and test accuracy as iterations increase.







The accuracy improves as iterations increase, indicating that the model spends sufficient amount of time over training and better captures intra-class variations. However, at K=10 the accuracy reaches its maximum value and it diminishes at K=20.

5 Generative Classification and Reconstruction with Missing Pixels

This part extends the generative classifier to handle test images with missing pixels. The models trained earlier have been reused (as suggested in the assignment). Only the test images are censored, while the training images remain unchanged.

Let a test image $\mathbf{x}^t$ be divided into observed and missing parts:

$$\mathbf{x}^t = \begin{bmatrix} \mathbf{x}_o^t \\ \mathbf{x}_m^t \end{bmatrix}, \quad \mathbf{x}_o^t \in \mathbb{R}^{d-k},\ \mathbf{x}_m^t \in \mathbb{R}^k.$$

Here, the subscript o denotes observed pixels and m denotes missing ones.

Classification. For a class c, the marginal probability over the observed pixels is used for prediction:

$$P(\mathbf{x}_o^t \mid y^t = c, \boldsymbol{\theta}) = \int_{\mathbb{R}^k} P([\mathbf{x}_o^t, \mathbf{v}] \mid y^t = c, \boldsymbol{\theta})\, d\mathbf{v}.$$

If $\mathbf{x} \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$, then the marginal over observed coordinates is (since marginal of a joint Gaussian is also a Gaussian)

$$\mathbf{x}_o \sim \mathcal{N}(\boldsymbol{\mu}_o, \boldsymbol{\Sigma}_{oo}).$$

The predicted label is obtained using (just replacing $\mathbf{x}$ by $\mathbf{x}_o$ in the equation used for classifying images with no missing pixels)

$$\hat{y} = \arg\max_c \big[\log P(\mathbf{x}_o \mid y = c) + \log P(y = c)\big].$$

The implementation uses the same `predictGen()` function, modified to compute log-likelihoods only for observed pixels.

Reconstruction. Once the most likely class $\hat{y}$ is predicted, the missing pixels are reconstructed using the conditional mean of a Gaussian:

$$\hat{\mathbf{x}}_m = \boldsymbol{\mu}_m^{\hat{y}} + \boldsymbol{\Sigma}_{mo}^{\hat{y}} \big(\boldsymbol{\Sigma}_{oo}^{\hat{y}}\big)^{-1} \big(\mathbf{x}_o - \boldsymbol{\mu}_o^{\hat{y}}\big).$$

The true conditional expectation of the missing pixels is given by the full mixture expression:

$$p(\mathbf{x}_m \mid \mathbf{x}_o, y = \hat{y}) = \sum_{k=1}^{K} \underbrace{\frac{\pi_k^{\hat{y}} \mathcal{N}\Big(\mathbf{x}_o \mid \boldsymbol{\mu}_{k,o}^{\hat{y}}, \boldsymbol{\Sigma}_{k,oo}^{\hat{y}}\Big)}{\sum_{j=1}^{K} \pi_j^{\hat{y}} \mathcal{N}\Big(\mathbf{x}_o \mid \boldsymbol{\mu}_{j,o}^{\hat{y}}, \boldsymbol{\Sigma}_{j,oo}^{\hat{y}}\Big)}}_{\gamma_k(\mathbf{x}_o)} \mathcal{N}\Big(\mathbf{x}_m \mid \boldsymbol{\mu}_{k,m|o}^{\hat{y}}, \boldsymbol{\Sigma}_{k,m|o}^{\hat{y}}\Big),$$

However, evaluating this is computationally expensive. Hence, we approximate this as following:

$$k^* = \arg\max_k \pi_k^{\hat{y}} \mathcal{N}\left(\mathbf{x}_o \mid \boldsymbol{\mu}_{k,o}^{\hat{y}}, \boldsymbol{\Sigma}_{k,oo}^{\hat{y}}\right),$$

and apply the same conditional reconstruction formula using parameters $\{\boldsymbol{\mu}_{k^*}^{\hat{y}}, \boldsymbol{\Sigma}_{k^*}^{\hat{y}}\}$ (essentially ignoring the denominator in the original formula and just taking the maximum among all numerators).

Implementation Details.

- The function `censorImages()` is used to zero out a fixed central region of each test image (about 21% of pixels).
- The `predictGen()` function is reused with a mask argument to handle only visible pixels.
- The `reconstructImages()` function reconstructs censored regions using the Gaussian conditional mean.

The test accuracy on missing-pixel data is reported for each K, and censored, reconstructed, and true images are visualized side by side. Correct classifications usually correspond to visually accurate reconstructions, while incorrect ones often show mismatched or distorted reconstructions.

This implementation follows the lecture concept of using marginal and conditional properties of multivariate Gaussians to perform both classification and feature imputation.

5.1 Results on Censored Data

The test accuracy with censored pixels is plotted against K:

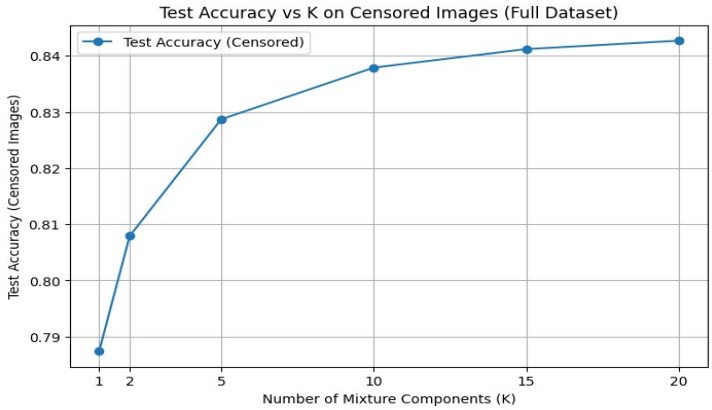


Figure 1: Test accuracy on censored MNIST images for different numbers of mixture components (K) with EM iterations = 10.

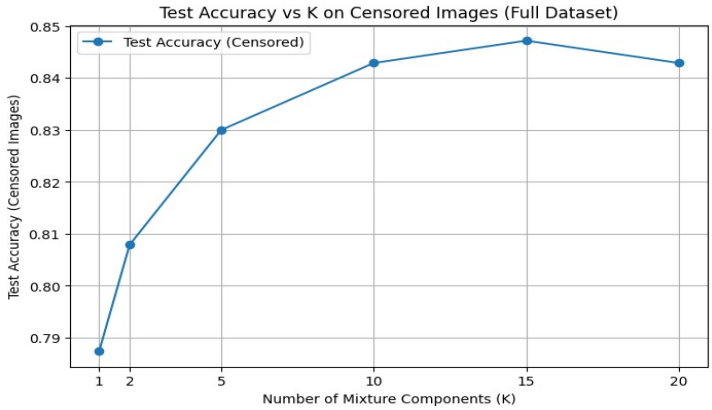


Figure 2: Test accuracy on censored MNIST images for different numbers of mixture components (K) with EM iterations = 20.

5.2 Classification and Reconstruction of Images

5.2.1 Unsuccessful and successful classification of images

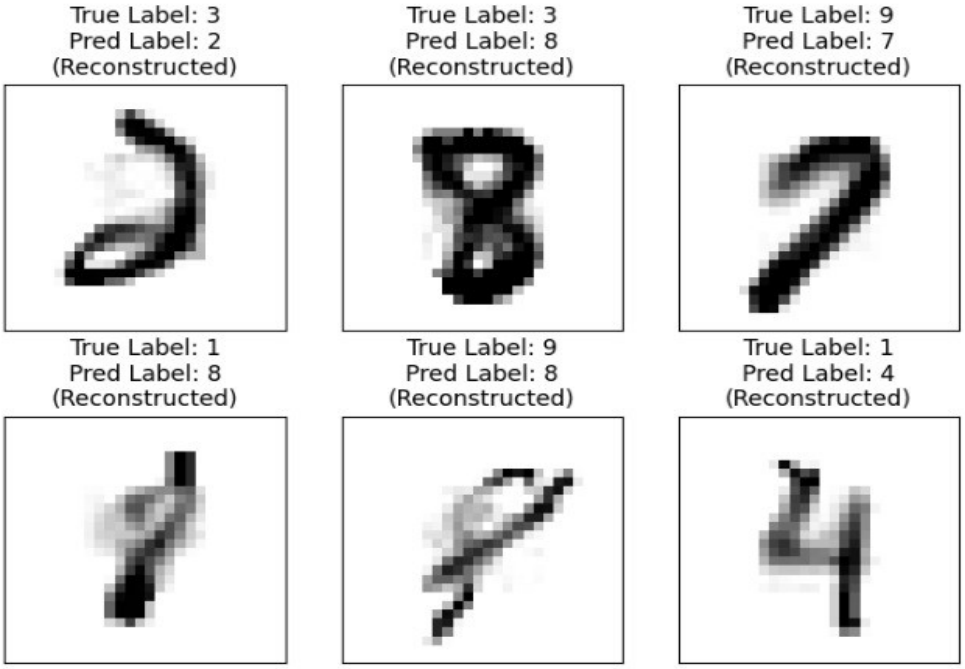


Figure 3: Unsuccessful classification of images

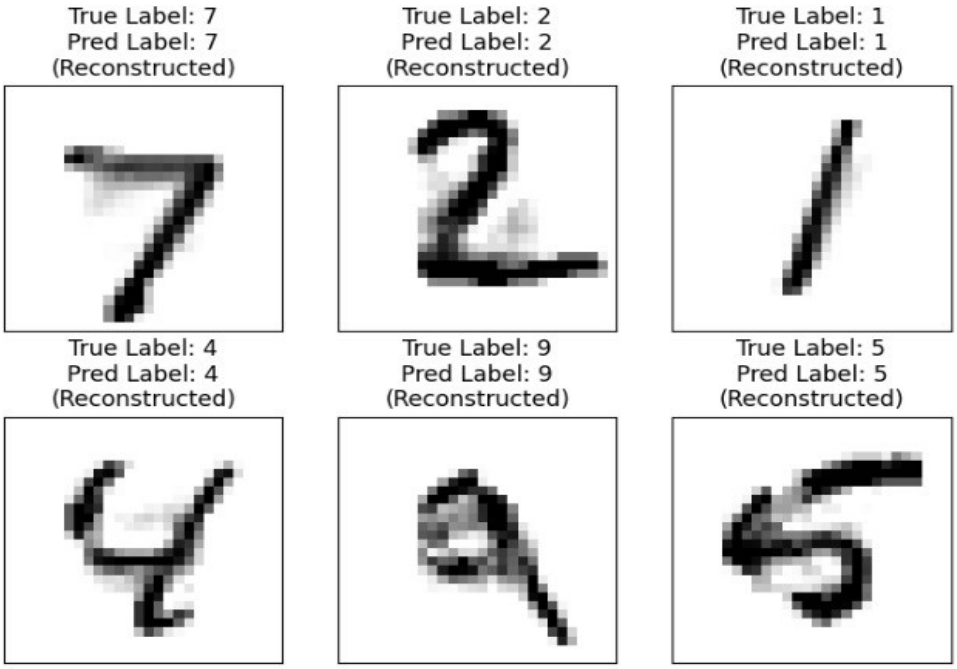


Figure 4: Successful classification of images

5.2.2 Unsuccessful and successful reconstruction of images

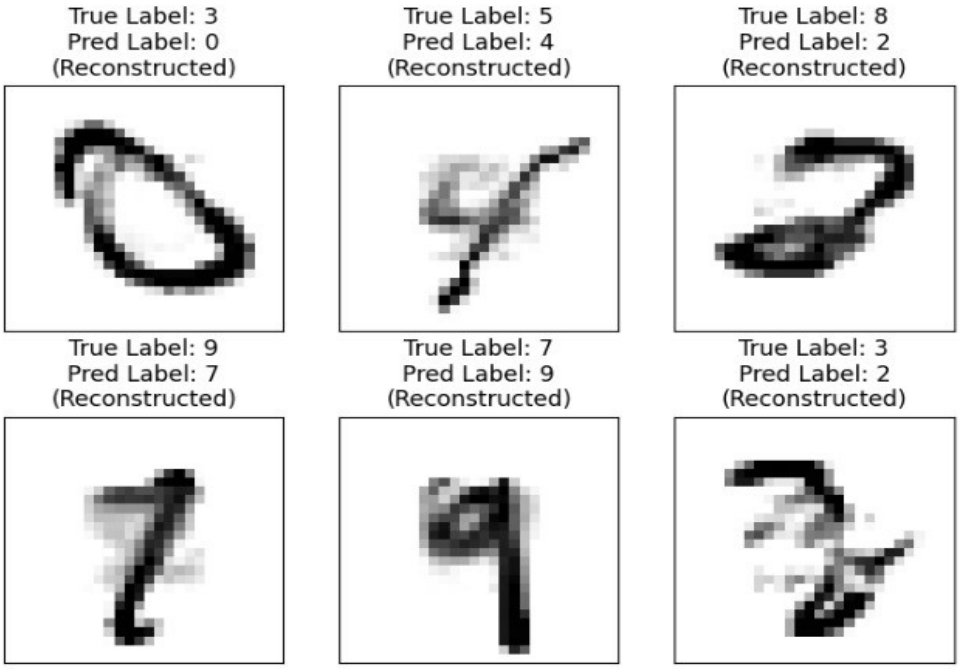


Figure 5: Unsuccessful reconstruction of images

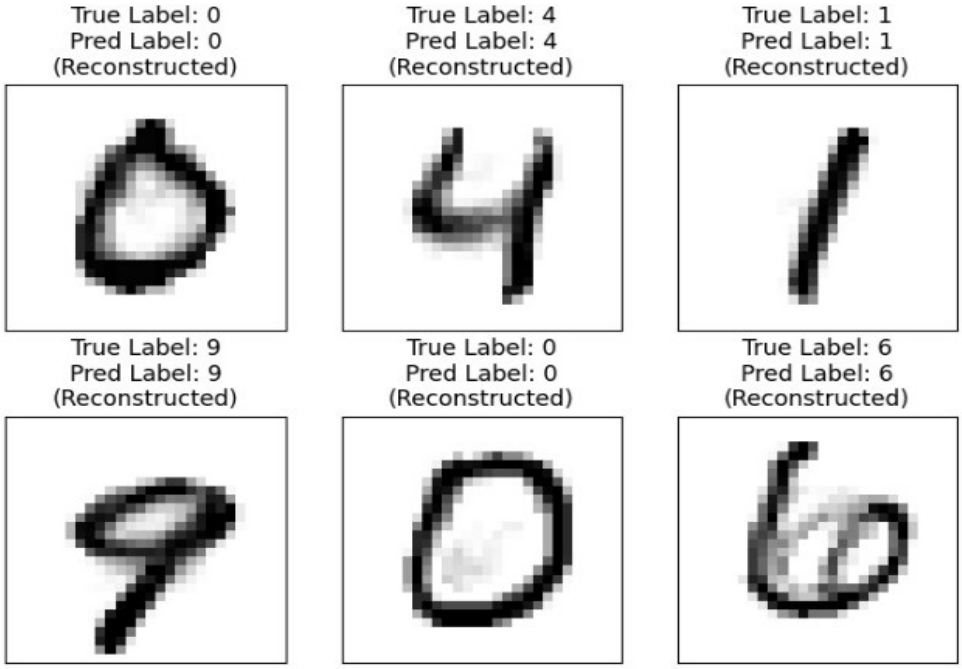


Figure 6: Successful reconstruction of images

As K increases, the model's ability to infer missing pixels improves, leading to smoother and more realistic reconstructions.

6 Analysis of Per-Class Accuracy Trends

Figure 7 shows the variation of training and test accuracies for each digit class as the number of Gaussian components K is increased ($K \in \{1, 2, 5, 10, 15, 20\}$). The blue curve corresponds to training accuracy, while the orange dashed curve corresponds to test accuracy.

For all digits, increasing K improves the training accuracy since additional mixture components allow the model to learn and accomodate for more variations in the writing style. However, test accuracy improves only up to a certain point just as observed earlier also since increasing K beyond a certain extent (different for different classes) results in overfitting and hence a poor accuracy for the testing part.

Digit-Wise Observations.

- **Digits 1, 7:** Test accuracy saturates early, typically by $K = 6 - 7$. These digits exhibit simple, consistent structures that can be well-captured by a single Gaussian or very few mixture components.
- **Digits 0, 2, 4, 6, 9 :** These classes show significant gains from increasing K up to 10-12, reflecting substantial intra-class diversity. Beyond this, the performance flattens or slightly drops due to overfitting. These digits have a little complexity in their structure which is captured perfectly by K = 10-12 .
- **Digits 3, 5, 8:** These digits are more complex in their writing style and perform best with larger K values like K=15.

Key Insights. Simpler digits such as 1 and 7 can be modeled effectively using $K = 6$ or 7, while more complex digits such as 0, 2, 4, 6, and 9 benefit from larger mixture counts ($K = 10$ to 12). Beyond $K \approx 12$, additional components mainly improve training accuracy but do not generalize well. This demonstrates that the optimal number of mixture components should be class-dependent.

Conclusion. Different digits exhibit distinct structural complexity levels. Classes with simple, well-defined shapes require fewer Gaussian components, whereas those with multiple stylistic variations benefit from richer mixture models. Therefore, assigning a uniform number of components to all classes is suboptimal; choosing class-specific K_c values can achieve comparable or improved test performance with fewer overall parameters.

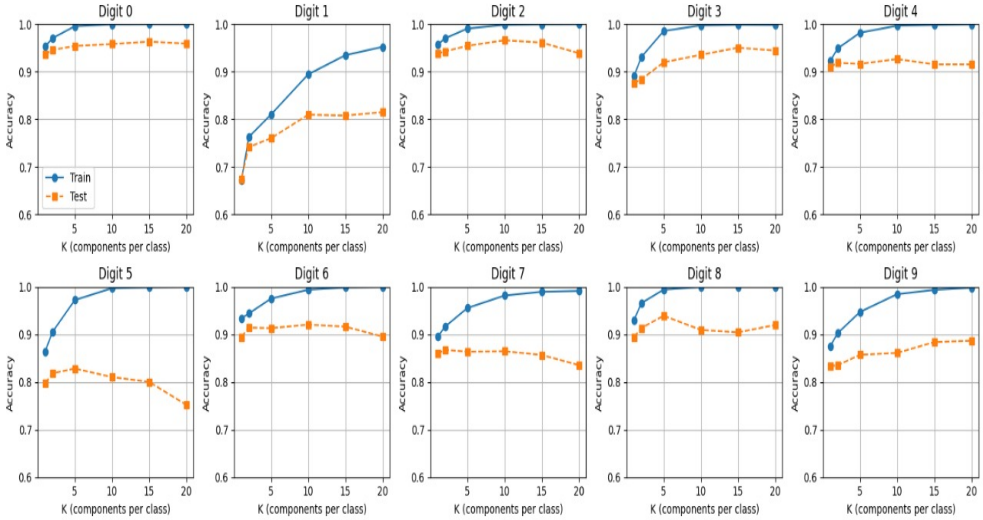


Figure 7: Digit-wise(per class) train and test accuracies vs K

References

[1] University of Toronto (CSC411/2515), *Gaussian Mixture Models and EM (Lecture 15/16 handout)*, available at: `https://www.cs.toronto.edu/~jlucas/teaching/csc411/lectures/lec15_16_handout.pdf`

[2] C. M. Bishop, *Gaussianisation&Marginals / Conditionals of the Multivariate Gaussian*, University of Pennsylvania, available at: https://www.seas.upenn.edu/~cis520/papers/Bishop_9.pdf

[3] "Comparing Gaussian mixture models with and without regularization." StackExchange (2017). https://stats.stackexchange.com/questions/293909/comparing-gaussian-mixture-models-with-and-without-regularization

[4] IIT Kanpur, *CS771: Introduction to Machine Learning – Lecture 13 & 14*, available at: https://www.cse.iitk.ac.in/users/purushot/courses/ml/2025-26-a/lectures.html

[5] IIT Kanpur, *CS771: Introduction to Machine Learning – Optimizations for reducing the required computation for EM provided in the code for Lecture 13*, available at: https://www.cse.iitk.ac.in/users/purushot/courses/ml/2025-26-a/lectures.html